Ingeniería en Tecnologías de la Información y las Comunicaciones

Tecnologías Inalámbricas

12:00 p m

ESP32 Bluetooth LE

Profesor: Ricardo Alejandro Rodríguez Jiménez

Equipo:

Ramírez Rodríguez Martín de Jesús 23151193

Santacruz Rodarte Cristian Alexis 22150108

Pedroza Pérez Emmanuel 23151202

Fecha: 22 de febrero de 2026

Índice

Introducción 3

MIT App Inventor 4

Circuito 7

Código ESP32 9

Conclusiones 20

Enlace de Repositorio: 21

Introducción

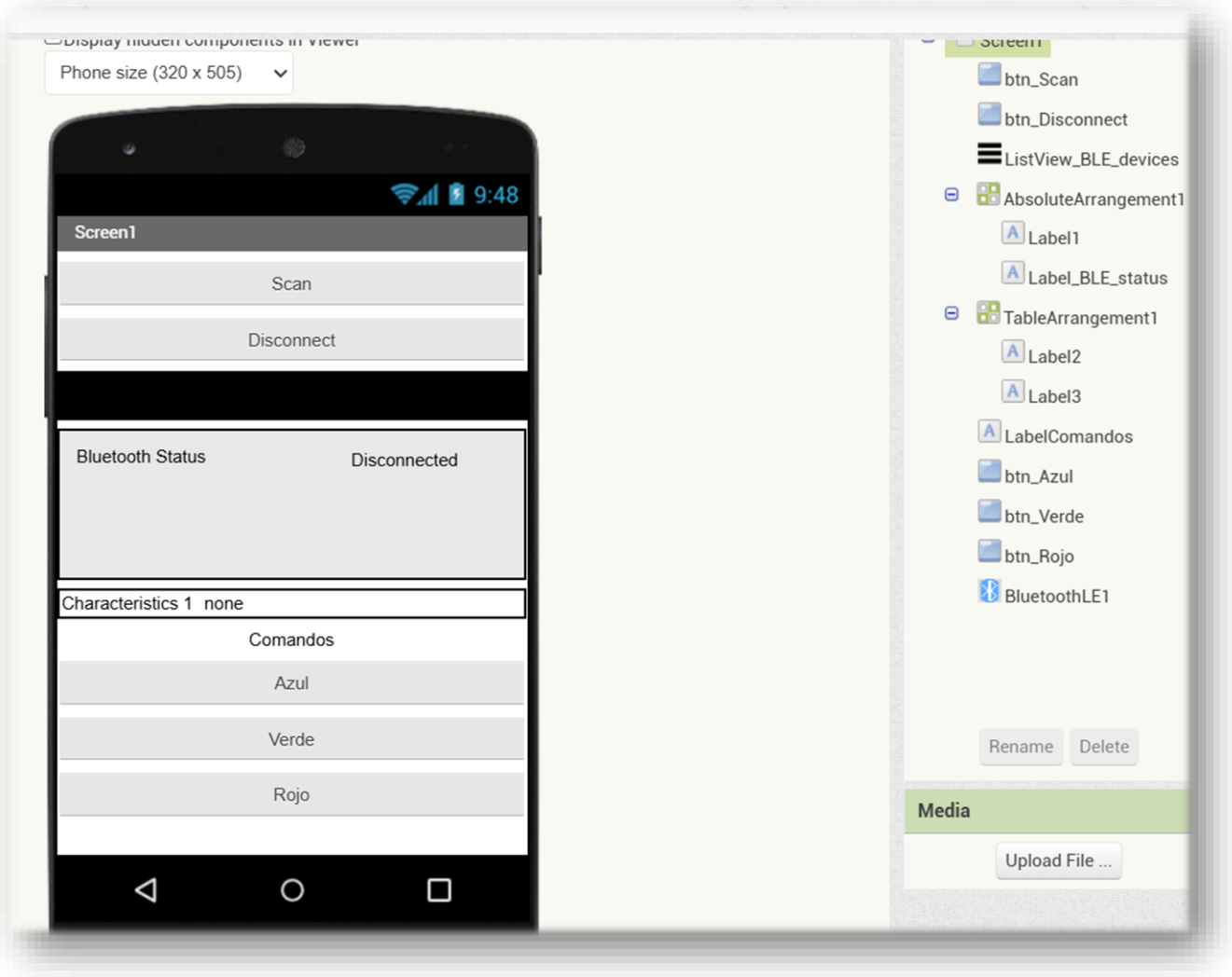
En el presente proyecto se desarrolla un sistema de control de iluminación basado en el microcontrolador ESP32, aprovechando sus capacidades de comunicación mediante Bluetooth Low Energy (BLE). El objetivo principal es establecer un enlace inalámbrico entre el dispositivo embebido y una aplicación móvil diseñada en MIT App Inventor, con el fin de manipular de manera remota el encendido y cambio de color de un conjunto de LED's RGB.

La implementación combina tres elementos fundamentales:

- Hardware: un circuito sencillo con el ESP32 y resistencias limitadoras para cada canal de los LEDs, garantizando un manejo seguro de la corriente.
- Software embebido: un programa en C++ que configura el servidor BLE, define servicios y características, y procesa los comandos enviados desde la aplicación móvil para modificar el color de los LEDs.
- Aplicación móvil: una interfaz gráfica intuitiva creada en MIT App Inventor, que solicita permisos de conexión, escanea dispositivos disponibles y envía instrucciones específicas ("R", "G", "B") al ESP32.

Este proyecto ejemplifica la integración de tecnologías de comunicación inalámbrica con sistemas de control electrónico, mostrando cómo el ESP32 puede convertirse en un puente eficiente entre el mundo físico y las aplicaciones móviles. Además, permite explorar conceptos clave como la gestión de permisos en Android, la programación orientada a eventos y el uso de UUIDs para identificar servicios y características en BLE.

MIT App Inventor



Captura de pantalla desde la página web de MIT App Inventor, mostrando la pantalla de la aplicación a usar.

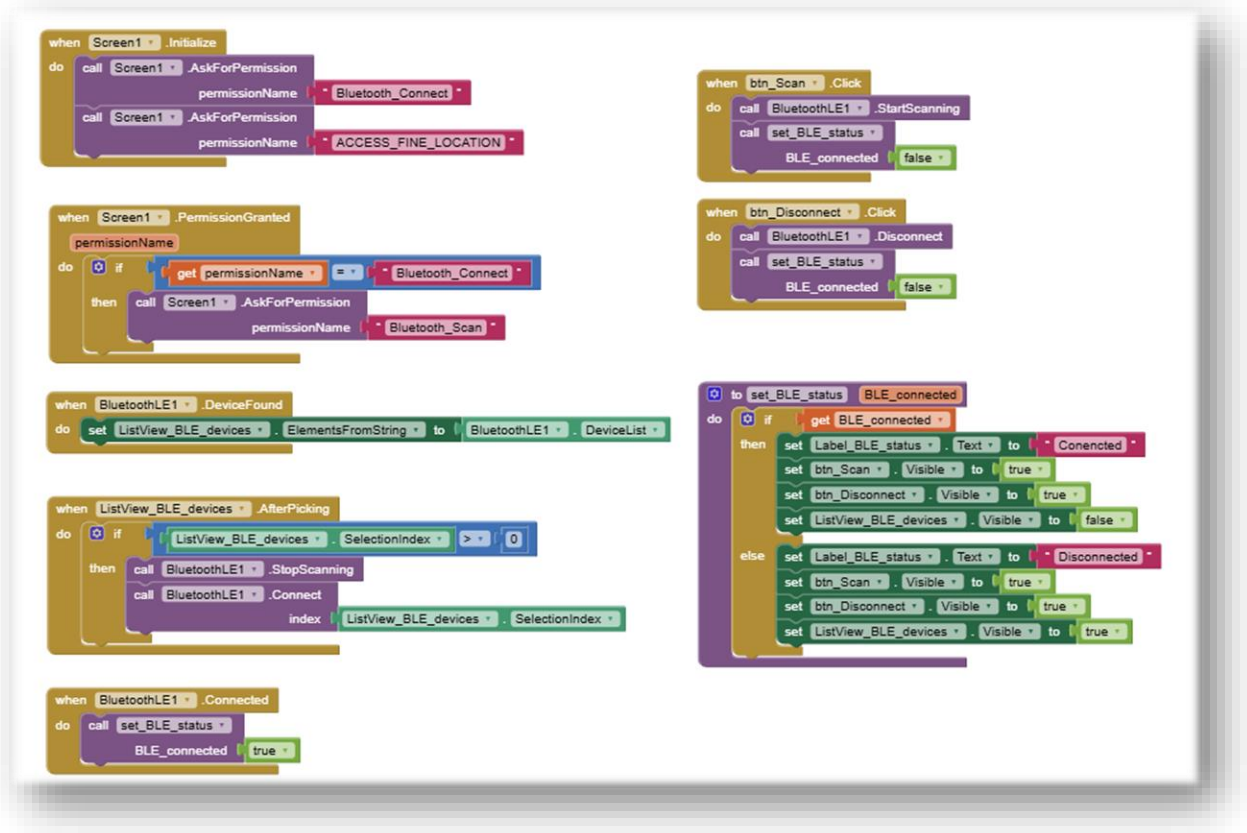
Al momento de desarrollar la aplicación para el teléfono y permitir su conexión a dispositivos por BLE, se completó de acuerdo con los recursos gráficos vistos en clase mediante etiquetas, botones y una lista, así como una extensión para la página que habilite las funciones BLE.

Esta primera etapa fue sencilla e intuitiva:

- Se añadió un botón Scan.
- Se añadió un botón Disconnect.
- Se añadió una pantalla de lista para mostrar los dispositivos que detecte.
- Se añadieron cuatro etiquetas para después mostrar el dispositivo conectado.

- Se añadieron tres botones para mandar instrucciones al ESP32 una vez que se haya conectado.

A continuación, se desarrolló la parte de bloques de la aplicación, donde hubo más inconvenientes, pues encontrar algunos bloques se volvió complicado al ser la primera interacción con esta página, especialmente al tener que crear los bloques de BLE conectado donde el proceso debía crearse de una manera fuera de lo usual en nuestra experiencia.



Captura de pantalla desde la página web de MIT App Inventor, mostrando la pantalla de la bloques de comportamiento de la aplicación a usar.

En un inicio se había recreado el código que se entregó en clase, sin embargo, se experimentaron varios errores de permisos con la aplicación, donde pedía permisos de ubicación y de conexión aunque se hayan otorgado desde el teléfono; problemas con la muestra de la lista de los dispositivos escaneados y conectados, pues la lista no arrojaba resultados tras iniciar el escáner y no dejaba interrumpir el proceso ni seguir con el mismo.

A diferencia del código mostrado en clase, esta aplicación fue modificada para que funcione con los estándares de la actualidad, pues se han agregado más permisos necesarios como el

“access file location” y así permitir el acceso a la ubicación del dispositivo. Adicionalmente, hubo un conflicto con las etiquetas lógicas al momento de mostrar o no mostrar ciertos criterios que corrían en la aplicación pero al darle un poco de tanteo todo terminó funcionando adecuadamente.

Captura de pantalla desde la página web de MIT App Inventor, mostrando la pantalla de la bloques de comportamiento de la aplicación a usar.

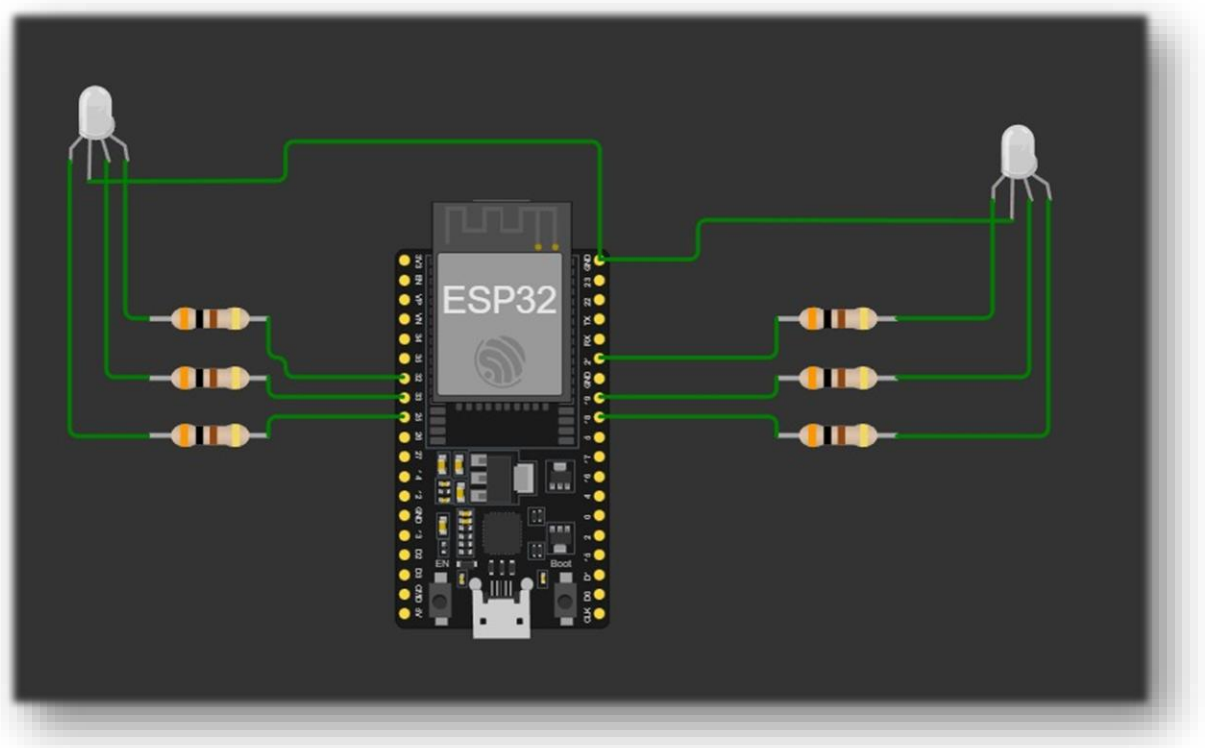
Tras comprobar que la primera parte funcionara, se agregaron las funciones para los botones de los colores y hacer que el ESP32 cambie un segundo LED RGB según la instrucción de recibir una letra acorde a la inicial del color solicitado en inglés. Según estos bloques, se confirma que se haya establecido conexión con el dispositivo correcto, considerando que los ‘UUID’ y ‘CharacteristicUUID’ sean los que corresponden con el ESP32; finalmente envía una lista de strings de ‘R’, ‘G’ o ‘B’ para que el ESP32 reciba las letras y actúe en consecuencia.

Una vez completada la aplicación en su totalidad, se construyó su ‘.apk’ para Android.

En el primer teléfono no funcionó la aplicación, siendo Samsung Galaxy s22 Ultra y teniendo sus respectivos permisos, no era capaz de encontrar el dispositivo ESP32.

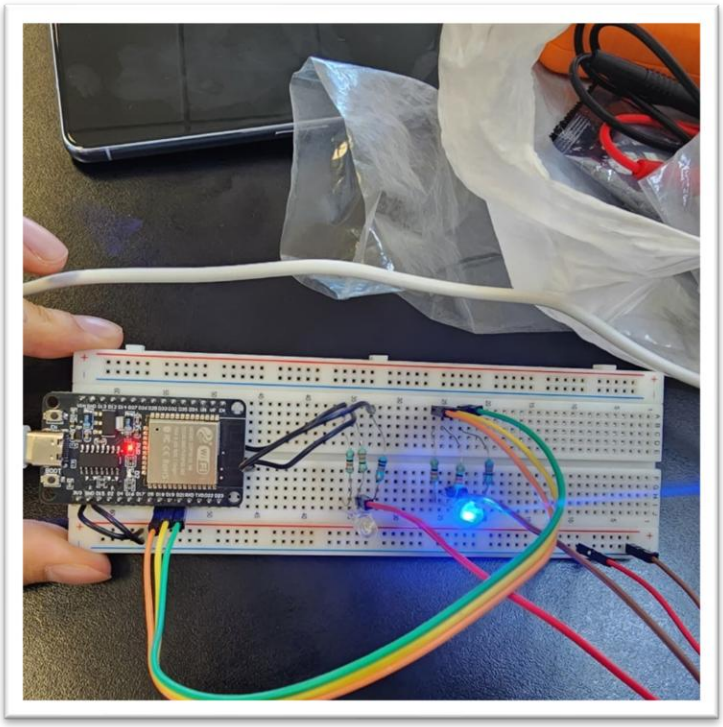
Se probó con un segundo teléfono, un Samsung s21 Ultra y la misma aplicación ahora fue capaz de utilizarse. El teléfono ahora puede encontrar dispositivos BLE y encontrar el ESP32 del equipo.

Circuito

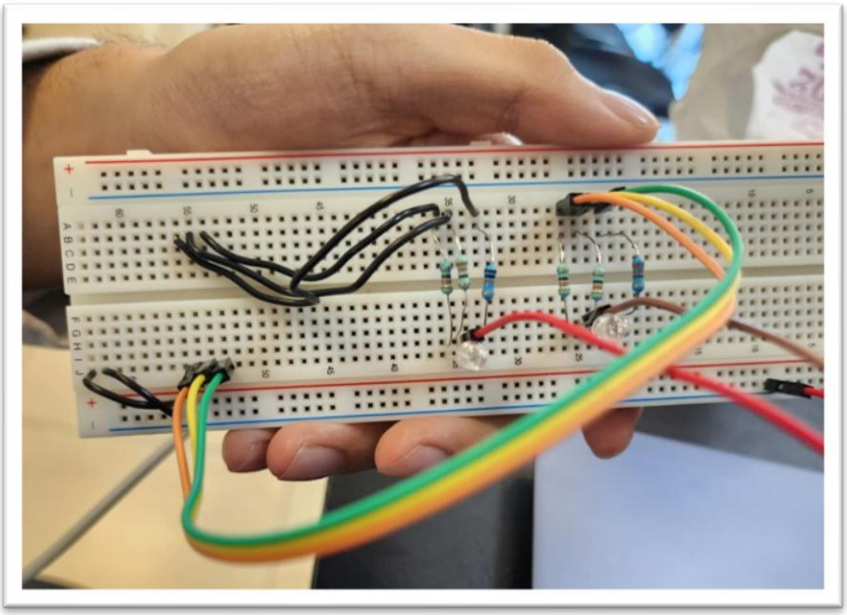


Este fue el circuito utilizado, enlazando un LED al ESP32 con el fin de mostrar el estado de conexión del mismo con la aplicación; y un segundo LED para que el ESP32 mande sus instrucciones de acuerdo a la aplicación.

Los problemas enfrentados en esta fase fueron las conexiones con los LEDs, pues éstos no encendían a pesar de la funcionalidad del código y del microcontrolador. Por lo tanto, se optó con probar nuevos focos LED y tampoco funcionaron, por lo que se descartó que fueran éstos y se optó por cambiar los cables a cables sencillos pelados, de esta manera ya se estableció el correcto flujo de energía entre los componentes.



Fotografía del circuito utilizado; tomado en el Centro de Información del ITA el día 20 de febrero de 2026 a las 13:05 hrs.



Fotografía del circuito utilizado; tomado en el Centro de Información del ITA el día 20 de febrero de 2026 a las 13:05 hrs.

Código ESP32

ESP32_BLE_controller.ino

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

#define DEVICE_NAME "No se conecten" // Nombre del dispositivo
#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_UUID    "beb5483e-36e1-4688-b7f5-ea07361b26a8"
#define BRIGHTNESS   255

// Led estado de conexion
#define PIN_RED   21
#define PIN_GREEN 19
#define PIN_BLUE  18

// Led controlado
int pinR = 25;
int pinG = 33;
int pinB = 32;

BLEServer* pServer = NULL;
BLECharacteristic* pCharacteristic = NULL;
bool deviceConnected = false;
bool oldDeviceConnected = false;

// Led estado
void rgbWrite(int r, int g, int b) {
  analogWrite(PIN_RED, r);
  analogWrite(PIN_GREEN, g);
  analogWrite(PIN_BLUE, b);
}

void flashLED(int r, int g, int b, int count) {
  for (int i = 0; i < count; i++) {
```

ESP32_BLE_controller.ino

```
    rgbWrite(r, g, b); delay(100);
    rgbWrite(0, 0, 0); delay(100);
  }
}

// Led controlado
void setColor(char c) {
  digitalWrite(pinR, LOW);
  digitalWrite(pinG, LOW);
  digitalWrite(pinB, LOW);

  if (c == 'R') digitalWrite(pinR, HIGH);
  if (c == 'G') digitalWrite(pinG, HIGH);
  if (c == 'B') digitalWrite(pinB, HIGH);
}

// Callbacks BLE
class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) { deviceConnected = true; };
    void onDisconnect(BLEServer* pServer) { deviceConnected = false; }
};

// Callback letra app
class MyCallbacks: public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
      String value = pCharacteristic->getValue();
      if (value.length() > 0) {
        Serial.print("Dato recibido: ");
        Serial.println(value);
        setColor(value.charAt(0));
      }
    }
};
```

ESP32_BLE_controller.ino

```
69
70
71  void setup() {
72    Serial.begin(115200);
73    Serial.println("Iniciando BLE...");
74
75    pinMode(pinR, OUTPUT);
76    pinMode(pinG, OUTPUT);
77    pinMode(pinB, OUTPUT);
78
79    BLEDevice::init(DEVICE_NAME);
80    pServer = BLEDevice::createServer();
81    pServer->setCallbacks(new MyServerCallbacks());
82
83    BLEService *pService = pServer->createService(SERVICE_UUID);
84
85    pCharacteristic = pService->createCharacteristic(
86                        CHAR_UUID,
87                        BLECharacteristic::PROPERTY_READ |
88                        BLECharacteristic::PROPERTY_WRITE |
89                        BLECharacteristic::PROPERTY_NOTIFY
90                      );
91    pCharacteristic->addDescriptor(new BLE2902());
92    pCharacteristic->setCallbacks(new MyCallbacks());
93
94    pService->start();
95
96    BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
97    pAdvertising->addServiceUUID(SERVICE_UUID);
98    pAdvertising->setScanResponse(true);
99    pAdvertising->setMinPreferred(0x12);
100   BLEDevice::startAdvertising();
101
102   Serial.println("Esperando cliente...");
103 }
```

```
void loop() {
  // led estado
  if (deviceConnected && !oldDeviceConnected) {
    oldDeviceConnected = deviceConnected;
    Serial.println(">>> CONECTADO <<<");
    flashLED(0, BRIGHTNESS, 0, 5);
  }

  if (!deviceConnected && oldDeviceConnected) {
    Serial.println(">>> DESCONECTADO <<<");
    flashLED(BRIGHTNESS, 0, 0, 5);
    delay(500);
    pServer->startAdvertising();
    oldDeviceConnected = deviceConnected;
  }

  if (deviceConnected) {
    rgbWrite(0, BRIGHTNESS, 0);
  } else {
    rgbWrite(0, 0, BRIGHTNESS);
  }

  delay(10);
}
```

A la par del desarrollo de la aplicación, se creó y adaptó el código que también se estudió en clase para que el ESP32 del equipo pueda comportarse como un dispositivo BLE.

Se importan las librerías necesarias para las funciones Bluetooth LE.

#include <BLEDevice.h>

#include <BLEServer.h>

#include <BLEUtils.h>

#include <BLE2902.h>

Definimos como se llamará el dispositivo y agregamos los identificadores de BLE además de el brillo máximo que puede tener cualquiera de los dos LED.

```
#define DEVICE_NAME "No se conecten" // Nombre del dispositivo

#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"

#define CHAR_UUID    "beb5483e-36e1-4688-b7f5-ea07361b26a8"

#define BRIGHTNESS   255
```

Definimos que pines del ESP-32 serán los que controlarán los respectivos pines de los dos LEDs RGB.

```
// Led estado de conexion

#define PIN_RED   21

#define PIN_GREEN 19

#define PIN_BLUE  18

// Led controlado

int pinR = 25;

int pinG = 33;

int pinB = 32;
```

Se declaran dos punteros, pServer y pCharacteristic que apuntan a objetos necesarios para la comunicación BLE.

```
BLEServer* pServer = NULL;

BLECharacteristic* pCharacteristic = NULL;
```

Variables de tipo bool que guardan el estado de la conexión y si un dispositivo anterior está conectado todavía o si fue desconectado.

```
bool deviceConnected = false;

bool oldDeviceConnected = false;
```

Función para que el ESP-32 pueda controlar el LED RGB que indica el estado de la conexión BLE.

```
// Led estado

void rgbWrite(int r, int g, int b) {

  analogWrite(PIN_RED, r);

  analogWrite(PIN_GREEN, g);

  analogWrite(PIN_BLUE, b);

}
```

Función que hace parpadear el LED para el estado.

```
void flashLED(int r, int g, int b, int count) {

  for (int i = 0; i < count; i++) {

    rgbWrite(r, g, b); delay(100);

    rgbWrite(0, 0, 0); delay(100);

  }

}
```

Apaga el LED y después, según la letra que indica el color que reciba, hace al respectivo pin del ESP-32 de activarse al máximo.

```
// Led controlado
void setColor(char c) {
  digitalWrite(pinR, LOW);
  digitalWrite(pinG, LOW);
  digitalWrite(pinB, LOW);

  if (c == 'R') digitalWrite(pinR, HIGH);
  if (c == 'G') digitalWrite(pinG, HIGH);
  if (c == 'B') digitalWrite(pinB, HIGH);
}
```

Esta clase permite saber si actualmente hay algún dispositivo conectado.

```
// Callbacks BLE
class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) { deviceConnected = true; };
    void onDisconnect(BLEServer* pServer) { deviceConnected = false; }
};
```

Esta clase utiliza onWrite para que solo sea ejecutada si recibe el valor char de una letra. Después de obtener el valor, llama al método setColor para que encienda el LED RGB del color solicitado.

```
// Callback letra app
class MyCallbacks: public BLECharacteristicCallbacks {
  void onWrite(BLECharacteristic *pCharacteristic) {
    String value = pCharacteristic->getValue();
    if (value.length() > 0) {
      Serial.print("Dato recibido: ");
      Serial.println(value);
      setColor(value.charAt(0));
    }
  }
};
```

Configuración de la comunicación serial y de los pines del LED.

```
Serial.begin(115200);
Serial.println("Iniciando BLE...");
pinMode(pinR, OUTPUT);
pinMode(pinG, OUTPUT);
pinMode(pinB, OUTPUT);
```

Indica que se inicie el dispositivo BLE con el nombre asignado.

```
BLEDevice::init(DEVICE_NAME);
```

Indica que se haga un servidor BLE en el ESP32, además se le asigna la función de Callback para comprobar conexiones y desconexiones.

```
pServer = BLEDevice::createServer();
pServer->setCallbacks(new MyServerCallbacks());
```

Indica que se haga el servicio BLE con el UUID asignado.

```
BLEService *pService = pServer->createService(SERVICE_UUID);
```

Realiza una característica o variable para que se pueda leer, escribir y además que notifique cuando el valor de la característica cambie.

```
pCharacteristic = pService->createCharacteristic(
                CHAR_UUID,
                BLECharacteristic::PROPERTY_READ |
                BLECharacteristic::PROPERTY_WRITE |
                BLECharacteristic::PROPERTY_NOTIFY
              );
```

Añade el descriptor BLE2902 para permitir las notificaciones y se asigna el método MyCallbacks() para controlar el LED.

```
pCharacteristic->addDescriptor(new BLE2902());
pCharacteristic->setCallbacks(new MyCallbacks());
```

Activa el servicio BLE.

```
pService->start();
```

Inicia el ‘advertising’ del dispositivo BLE añadiendo el UUID, para que pueda responder al escaneo y se agrega un parámetro que mejora la conexión.

```
  BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  pAdvertising->setScanResponse(true);
  pAdvertising->setMinPreferred(0x12);
```

El dispositivo comienza a anunciarse para poder ser encontrado.

```
  BLEDevice::startAdvertising();
  Serial.println("Esperando cliente...");
}
```

Revisa si hay un dispositivo que se acabe de conectar y no había una conexión anterior, cambia el estado del dispositivo anterior al conectado actualmente y manda instrucción al LED de estado para que indique que se conectó un dispositivo parpadeando 5 veces.

```
  // led estado
  if (deviceConnected && !oldDeviceConnected) {
    oldDeviceConnected = deviceConnected;
    Serial.println(">>> CONECTADO <<<");
    flashLED(0, BRIGHTNESS, 0, 5);
  }
```

Revisa si se desconectó el dispositivo, da la instrucción al LED para indicar la desconexión con un color rojo y reinicia la función de anuncio para que se pueda encontrar de nuevo.

```
  if (!deviceConnected && oldDeviceConnected) {
    Serial.println(">>> DESCONECTADO <<<");
    flashLED(BRIGHTNESS, 0, 0, 5);
    delay(500);
```

```
  pServer->startAdvertising();
  oldDeviceConnected = deviceConnected;
 }
```

Si no hay cambios en la conexión del dispositivo, si hay conexión mantiene un color verde y si no hay conexión mantiene un color azul.

```
 if (deviceConnected) {
  rgbWrite(0, BRIGHTNESS, 0);
 } else {
  rgbWrite(0, 0, BRIGHTNESS);
 }
 delay(10);
}
```

En cuando a este código realizado, no ocurrieron inconvenientes serios, pues solamente se requirió de adaptarlo conforme se corregía la aplicación.

Finalmente, este trabajo esperado para el jueves 19 de febrero de 2026 se retrasó por un día por tiempo y los inconvenientes de la aplicación, sin embargo, se completó con éxito el día 20 de febrero de 2026 en el Instituto Tecnológico de Aguascalientes a las 13:06pm. El trabajo volvió a tardarse debido a la falta de comunicación y logística que presentan los fines de semana.

Conclusiones

Ramírez Rodríguez Martín de Jesús:

El desarrollo de este proyecto permitió comprobar la versatilidad del ESP32 como plataforma de control y comunicación inalámbrica mediante Bluetooth Low Energy (BLE) y Bluetooth clásico. La integración con una aplicación móvil creada en MIT App Inventor demostró cómo es posible establecer una interacción sencilla y confiable entre el usuario y su hardware, logrando controlar de manera remota el encendido y cambio de color de LEDs RGB. A pesar de los problemas con la creación de la aplicación, el resultado fue satisfactorio y se encontró cierto gusto por la plataforma de MIT.

Esta experiencia evidenció la importancia de comprender tanto la configuración del servidor BLE y sus características, como la correcta gestión de permisos en la aplicación móvil. Asimismo, se resaltó el valor de reencontrarse con conocimientos de electrónica básica.

En conjunto, este trabajo constituyó un agradable primer ejemplo práctico de la convergencia entre hardware y software enfocados a seguridad, mostrando cómo herramientas accesibles son utilizadas para diseñar sistemas que acercan a los principios del Internet de las Cosas y al control remoto de dispositivos electrónicos.

Santacruz Rodarte Cristian Alexis:

La práctica realizada nos dejó bastante satisfechos con lo aprendido, ya que usamos tecnologías inalámbricas tales como programación con Arduino para las placas ESP32 y programación de aplicaciones con base en BLE.

Pedroza Pérez Emmanuel:

Veo necesarias este tipo de prácticas para familiarizarnos con la sintaxis y la estructura del código para utilizar BLE tanto para el Arduino como para la APP. Gracias a la práctica logramos entender de mejor manera cómo funciona el BLE permitiéndonos realizar dispositivos con funciones prácticas.

Enlace de Repositorio:

Aquí se encontrará un enlace para redirigirle al repositorio donde se presentan el código de ESP32, video de evidencias y la misma documentación:

https://github.com/MartinITAAgs/Tecnolog-as-Inal-mbricas-12pm/tree/main/esp32